Лабораторная работа №3

Скриптовый язык программирования PHP

Цель работы: ознакомиться с основными возможностями языка PHP, синтаксисом, основными операциями, получить практические навыки программирования.

Основные понятия

Для работы скрипта не требуется описывать какие-либо переменные, используемые модули, главную функцию и т. п. Любая программа может начинаться непосредственно с оператора PHP внутри специальных тегов.

Простейшая программа Hello world на PHP выглядит следующим образом:

```
<?php
  echo 'Hello, world!';
?>
```

Комментарии

- Однострочные //, #
- Многострочные /* … */

Типы данных

PHP поддерживает 8 примитивных типов.

4 скалярных типа:

- boolean
- integer
- число с плавающей точкой (float)
- string

Два составных типа:

- array
- object

И, наконец два специальных типа:

- resource
- NULL

Переменные

Декларируются знаком dollar ($) с последующим именем переменной. Имя переменной чувствительно к регистру символов. Правильное имя переменной начинается с буквы или

символа подчёркивания, с последующими (в любом количестве) буквами, числами или символами подчёркивания.

Предопределённые переменные

Серверные переменные: $_SERVER

$_SERVER это массив, содержащий такую информацию, как headers/шапки (передаваемые браузером), paths/пути и размещение скриптов. Ключи и их значения в этом массиве создаются web-сервером. Нет гарантии, что каждый web-сервер будет предоставлять что-нибудь из этого. Сервер может опустить некоторые позиции или добавить новые не стандартные.

HTTP-Куки: $_COOKIE

Ассоциативный массив переменных, передаваемых текущему скрипту через HTTP cookies. Автоматически глобальны в любой области видимости.

HTTP GET-Переменные: $_GET

Ассоциативный массив переменных, передаваемых текущему скрипту через метод HTTP GET. Автоматически глобальны в любой области видимости.

HTTP POST-Переменные: $_POST

Ассоциативный массив переменных, передаваемых текущему скрипту через метод HTTP POST. Автоматически глобальны в любой области видимости.

Переменные HTTP-загрузки файлов: $_FILES

Ассоциативный массив элементов, загружаемых в текущий скрипт методом HTTP POST. Автоматически глобальны в любой области видимости.

Переменные запроса: $_REQUEST

Ассоциативный массив, состоящий из содержимого `$_GET`, `$_POST`, `$_COOKIE` и `$_FILES`.

Переменные сессии: $_SESSION

Ассоциативный массив, содержащий переменные сессии, доступные текущему скрипту.

Глобальные переменные: $GLOBALS

Ассоциативный массив, содержащий ссылки на все переменные, которые определены в данный момент в глобальной области видимости скрипта. Имена переменных являются ключами в массиве.

Область видимости переменной

Область видимости переменной это контекст, в котором она определяется. Как правило все переменные PHP имеют единую область видимости. Эта единая область видимости включает также include и необходимые файлы.

В PHP существуют глобальные (доступны в локальной области видимости через global и $_CLOBALS), локальные (переменные функций) и статические переменные.

Константы в PHP

Константой называется именованная величина, которая не изменяется в процессе выполнения программы.

В PHP константы определяются функцией **define()**. Эта функция имеет следующий формат:

define (**$name**, **$value**, **$case_sen**), где:

$name - имя константы;
$value - значение константы;
$case_sen - необязательный параметр логического типа, указывающий, следует ли учитывать регистр букв (true) или нет (false).

Пример определения и использования констант в PHP:

```
<?php
    define("pi",3.14,true);
    echo pi; // Выводит 3.14
?>
```

Проверка существования констант

Для проверки существования константы можно использовать функцию **defined()**. Данная функция возвращает **true**, если константа объявлена. Например:

```
<?php
    define("pi",3.14,true); // Объявляем константу pi
    if (defined("pi")==true) echo "Константа pi объявлена!";
    // Скрипт выведет 'Константа pi объявлена!'
?>
```

Условный оператор

```
if (условие) {
      операторы
} else {
      операторы
}
```

Циклы

С предусловием

```
while (условие) {
        операторы
}
```

С постусловием

```
do {
   операторы
} while (условие);
```

С параметром

```
for (инициализация; условие; приращение) { операторы }
```

Совместный

```
foreach($array as $key => $value) { операторы }
```

Оператор break

Заканчивает выполнение текущей структуры for, foreach, while, do..while или switch.

Оператор continue

Используется в циклических структурах для пропуска оставшейся части текущей итерации цикла и продолжения выполнения с начала следующей итерации (прохода) цикла.

Оператор switch

Напоминает выполнение серии операторов IF над одним выражением. Во многих случаях вам нужно будет сравнить одну переменную (или выражение) с различными значениями и выполнить разные участки кода в зависимости от того, каково значение

```
switch (выражение) {
    case значение1:
        statement
        break;
    case значение2:
        statement
        break;
    case значение3:
        statement
        break;
}
```

Оператор return

Если вызывается из функции, оператор **return()** немедленно заканчивает выполнение текущей функции и возвращает свой аргумент как значение вызова функции.

Тернарная операция

```
$var = условие ? оператор1 : оператор2;
```

Операции подключения файлов

Данные операции включают файл в текущий и вычисляют его

- require()
- include()
- require_once()
- include_once()

Все эти операторы подключают и исполняют файл, т.е., по сути, подставляют текст из файла в программный код текущего скрипта. При этом к переменным подключаемого файла применяется та область видимости, которая была в строке вызова оператора подключения. Переменные, объявленные ранее, будут доступны в подключенном файле и наоборот.

Наиболее часто эти операторы применяются для того, чтобы подключать и использовать одинаковые участки кода в разных исполняемых файлах. Зачастую такими участками кода являются шаблоны или их составляющие, например шапка (header) и подвал (footer) сайта. Header содержит html- и php-код, который используется на страницах сайта до начала основного контента. Footer соответственно содержит код, который располагается после основного контента. Генерацию контента страницы осуществляет запрошенный пользователем исполняемый скрипт. Например:

Файл index.php

```
<?php
$title = 'Главная'; // объявляем заголовок страницы
$styles = array( // имена подключаемых файлов стилей
        'style.css', 'style1.css'
);
require 'header.php'; // подключаем шапку

echo "тут текст главной страницы"; // генерация контента

require 'footer.php'; // подключаем подвал
?>
```

Файл header.php

```
<?php header("Content-type: text/html;charset=utf-8"); ?>
<!DOCTYPE html>
<html>

<head>
  <title>
        <?php echo $title; /* выводим заголовок страницы */ ?>
  </title>
  <meta charset="utf-8" />
  <?php foreach ($styles as $style) { /* подключаем стили */  ?>
        <link rel="stylesheet" type="text/css" href="./css/<?=$style?>" />'
  <?php } ?>
</head>
<body>
        <div id="header">
                Шапка сайта кшрпмукшмрш
        </div>
        <div id="leftcol">
                Left Column
        </div>
        <div id="content"> <!-- открываем контейнер для основного контента -->
```

Файл footer.php

```
        </div> <!-- закрываем #content -->
</body>
</html>
```

Файл стилей style.css

```
#header {
        width: 100%;
        height: 200px;
        background-color: lightgreen;
}
#leftcol {
        float: left;
        width: 250px;
        min-height: 500px;
        background-color: lightblue;
}
#content {
        margin-left: 250px;
        min-height: 500px;
}
```

Файл стилей style1.css

```
#content {
        background-color: red;
}
```

Функции

Функция может быть определена с использованием такого синтаксиса:

```
function foo ($arg_1, $arg_2, ..., $arg_n) {
    echo "Пример \n";
    return $retval;
}
```

Передача аргументов по ссылке

```
function foo (&$arg1) {
      return;
}
```

Значения аргументов по умолчанию

```
function foo ($arg1 = "default_value") {
      return;
}
```

В качестве возвращаемого значения может быть любой тип данных

Функции переменных

PHP поддерживает концепцию функций переменных. Это значит, что, если имя переменной содержит скобки присоединённые к нему, PHP ищет функцию с тем же именем, в которое вычисляется переменная, и пытается выполнить её. Помимо прочего, это можно использовать для реализации callbacks/обратных вызовов, таблиц функций и так далее.

```
<?php
function foo() {
    echo "In foo()<br>\n";
}

function bar($arg = '') {
    echo "In bar(); argument was '$arg'.<br>\n";
}

$func = 'foo';
$func(); // вызов функции foo()
$func = 'bar';
$func('test'); // вызов функции bar('test')
?>
```

Классы и объекты

Класс это коллекция переменных и функций, работающих с этими переменными. Класс определяется с использованием следующего синтаксиса:

```
<?php
class className {
    var $property;  // свойства класса
    function method1 ($arg1, $arg2) { // метод method1
        $this->property [$arg1] = $arg2;
    }
    function method2 () { // метод method2
    {
        ...
```

```
        return false;
    }
}
?>
```

Создание класса на основе уже имеющегося:

```
class className1 extends className {
        ...
}
```

Конструкторы это функции класса, которые автоматически вызываются при создании нового экземпляра класса операцией new. Функция становится конструктором, когда она называется __construct() или имеет то же имя, что и класс, в котором она определена.

Иногда удобно обращаться к функциям или переменным в базовых классах или обратиться к функциям в классах, которые ещё не имеют экземпляров. Операция :: используется именно для этого.

```
class A {
    function example() {
        echo "I am the original function A::example().<br>\n";
    }
}

class B extends A {
    function example() {
        echo "I am the redefined function B::example().<br>\n";
        A::example();
    }
}
```

Ошибки в PHP

Ниже в таблице перечислены существующие в PHP ошибки и соответствующие им числовые значения

Значение	Константа	Описание
1	E_ERROR	фатальные ошибки времени выполнения
2	E_WARNING	предупреждения времени выполнения (нефатальные ошибки)
4	E_PARSE	ошибки разбора времени компиляции
8	E_NOTICE	уведомления времени выполнения (менее серьёзные, чем предупреждения)
16	E_CORE_ERROR	фатальные ошибки при начальном старте PHP
32	E_CORE_WARNING	предупреждения (нефатальные ошибки) при начальном старте PHP
64	E_COMPILE_ERROR	фатальные ошибки времени компиляции
128	E_COMPILE_WARNING	предупреждения времени компиляции (нефатальные ошибки)
256	E_USER_ERROR	генерируемое пользователем сообщение об ошибке

512	E_USER_WARNING	генерируемое пользователем предупреждение
1024	E_USER_NOTICE	генерируемое пользователем уведомление
	E_ALL	всё вышеуказанное, как поддерживаемое

Создание изображений и манипуляции с ними

В PHP нет ограничений на создание простого HTML-вывода. Он может также использоваться для создания изображений и манипуляций с файлами изображений различных форматов, включая gif, png, jpg. Что особенно удобно: php может выводить поток изображения непосредственно в браузер. Вам для этого необходимо скомпилировать PHP с библиотекой GD функций изображений. GD и PHP могут потребовать также наличия других библиотек, в зависимости от того, с каким форматом изображений вы работаете.

```
<?php
    Header("Content-type: image/png");
    $string=implode($argv," ");
    $im = ImageCreateFromPng("images/button1.png");
    $orange = ImageColorAllocate($im, 220, 210, 60);
    $px = (imagesx($im)-7.5*strlen($string))/2;
    ImageString($im,3,$px,9,$string,$orange);
    ImagePng($im);
    ImageDestroy($im);
?>
```

Куки/Cookies

Куки это механизм хранения данных в удалённом браузере и отслеживания и идентификации пользователей с их помощью. Вы можете установить куки функцией `setcookie()`. Куки являются частью HTTP-заголовков, поэтому `setcookie()` обязана вызываться до отправления любого вывода браузеру. Это то же самое ограничение, что и для `header()`. Кука имеет 6 параметров: имя, значение, домен, путь на домене, дата истечения и флаг безопасности. Имя и значение являются обязательными. При каждом запросе клиента какой-либо страницы браузер проверяет сохраненные куки, и в случае совпадения домена, пути и не наступлении даты истечения эта кука отправляется на сервер в формате имя=значение.

Любая кука, отправляемая серверу с клиента, будет автоматически конвертирована в PHP-переменную $_COOKIE["имя"] точно так же, как GET и POST-данные. Если вы хотите присвоить несколько переменных одной куке, просто добавьте [] в имя куки.

Загрузка файлов

PHP способен принимать загрузку файлов из любого соответствующего браузера. Это даёт возможность загружать текстовые и бинарные файлы.

```
<form enctype="multipart/form-data" action="_URL_" method="post">
        Send this file: <input name="userfile" type="file" />
        <input type="submit" value="Send File" />
</form>
```

URL должен указывать на PHP-скрипт для загрузки файла

После отправки формы на сервер содержимое массива `$_FILES` изменится. Обратите внимание, что здесь предполагается использование имени 'userfile' для загружаемого файла, как в примере скрипта ранее:

`$_FILES['userfile']['name']` - Оригинальное имя файла на клиентской машине.
`$_FILES['userfile']['type']` - mime-тип файла, если браузер предоставил эту информацию. Пример: "image/gif".
`$_FILES['userfile']['size']` - Размер загруженного файла в байтах.
`$_FILES['userfile']['tmp_name']` - Временное имя файла, под которым загруженный файл был сохранён на сервере.

Файлы будут по умолчанию сохраняться во временной директории на сервере, если только не задано другое место директивой upload_tmp_dir в php.ini.

PHP-скрипт для загрузки файла

```
<?php
if (is_uploaded_file($_FILES['userfile']['tmp_name'])) {
    copy($_FILES['userfile']['tmp_name'], "/place/to/put/uploaded/file");
} else {
    echo "Possible file upload attack. Filename: ".$_FILES['userfile']['name'];
}
/* или вместо copy
move_uploaded_file($_FILES['userfile']['tmp_name'],
"/place/to/put/uploaded/file"); */
?>
```

Файл будет удалён из временной директории по окончании запроса, если он (файл) не перемещён или не переименован.

Обработка передаваемых данных

Любые данные скрипту можно передавать методами get и post, в основном, при помощи отправки HTML-форм. Соответственно для обращения к этим данным необходимо использовать массивы $_GET и $_POST. Полученная информация должна быть обработана, проверена на наличие ошибок и использована по назначению. Пользователь обязательно

должен увидеть результаты использования этих данных. Если в процессе обработки или проверки были выявлены ошибки, то клиенту отправляется негативное сообщение с текстом этой ошибки. Иначе отправляется положительное сообщение об успешном выполнении операции. Также, в случае наличия ошибок, необходимо в поля формы вернуть значения, введенные пользователем, чтобы он привел их в соответствие требованиям, а не вводил заново. Обработка данных должна производиться до вывода в браузер html-текста. Результаты обработки необходимо сохранять в переменные, а затем выводить в том месте html-кода, где это нужно. Пример:

```
<?php
// обрабатываем данные до вывода в браузер
if (isset($_POST['btn_ok'])) { // если была нажата кнопка "Отправить"
   $err = array();
   if (strlen($_POST['login']) <= 3) {
         // сохраняем ошибку в переменную
         $err['login'] = 'Длина логина должна быть более 3-х символов';
   } else {
         header('Location: '.$_SERVER['PHP_SELF'].'?message=ok'); // делаем
редирект для предотвращения повторной отправки данных по нажатию F5 (обновить)
   }
}

require 'header.php';
if (isset($_GET['message']) && $_GET['message'] == 'ok')  // если был
редирект с сообщением OK
   echo '<p style="color: green;">Введенный вами логин корректен</p>';
?>
<form action="<?=$_SERVER['PHP_SELF']?>" method="post">
   <?php
         // выводим ошибку если она есть
         if (isset($err['login'])) echo '<p style="color:
red;">'.$err['login'].'</p>';
   ?>
   <label>Логин: </label><input type="text" name="login"
value="<?=(isset($_POST['login']) ? $_POST['login'] : '')?>" placeholder="Более
3-х символов" <?php if (isset($err['login'])) echo 'style="background-color:
red"'; ?> />
   <input type="submit" name="btn_ok" value="Отправить" />
</form>
<?php require 'footer.php';?>
```

Некоторые встроенные функции PHP

Математические функции

Константа	Значение	Описание
M_PI	3.14159265358979323846	Pi
M_E	2.7182818284590452354	e
M_LOG2E	1.4426950408889634074	log_2 e
M_LOG10E	0.43429448190325182765	log_10 e
M_LN2	0.69314718055994530942	log_e 2

M_LN10	2.30258509299404568402	log_e 10
M_PI_2	1.57079632679489661923	pi/2
M_PI_4	0.78539816339744830962	pi/4
M_1_PI	0.31830988618379067154	1/pi
M_SQRTPI	1.77245385090551602729	sqrt(pi)
M_SQRT2	1.41421356237309504880	sqrt(2)
M_SQRT3	1.73205080756887729352	sqrt(3)
M_SQRT1_2	0.70710678118654752440	1/sqrt(2)
M_LNPI	1.14472988584940017414	log_e(pi)

abs - абсолютное значение
acos - аккосинус
asin - акрсинус
atan - арктангенс
bindec - двоичное в 10-ричное
ceil - округление дробей в сторону увеличения
cos - косинус
decbin - десятеричное в двоичное
dechex - 10-ричное в 16-ричное
decoct - 10-ричное в 8-ричное
deg2rad - конвертирует число в градусах в эквивалент в радианах
exp - e в степени ...
floor - округляет дробь в сторону уменьшения
hexdec - 16-ричное в 10-ричное
is_nan - определяет, является ли переменная не числом
log10 - логарифм с базой 10
log - натуральный логарифм
max - находит наибольшее значение
min - находит наименьшее значение
mt_rand - генерирует наилучшее случайное значение
mt_srand - запускает генератор наилучшего случайного числа
number_format - форматирует число с группировкой по тысячам
octdec - 8-ричное в 10-ричное
pow - экспоненциальное выражение
rad2deg - конвертирует число в радианах в эквивалент в градусах
rand - генерирует случайное число
round - округляет число с плавающей точкой/float
sin - синус
sqrt - квадратный корень
srand - запускает генератор наилучшего случайного числа
tan - тангенс

Функции переменных

empty - определяет, установлена ли переменная
floatval - получает float-значение переменной
get_defined_vars - возвращает массив из всех определённых переменных
gettype - получает тип переменной
intval - получает целочисленное значение переменной

is_array - определяет, является ли переменная массивом
is_bool - определяет, является ли переменная boolean
is_float - определяет, является ли переменная числом с плавающей точкой
is_int - определяет, является ли переменная целочисленной
is_null - определяет, является ли переменная null
is_numeric - определяет, является переменная числом или строкой чисел
is_object - определяет, является ли переменная объектом
is_resource - определяет, является ли переменная ресурсом
is_string - определяет, является ли переменная строкой
isset - определяет, установлена ли переменная
print_r - печатает читабельную информацию о переменной
serialize - генерирует сохраняемое представление данного значения
strval - получает строковое значение переменной
unserialize - создаёт PHP-значение из хранимого представления
unset - отменяет установку/Unset данной переменной
var_dump - выполняет дамп информации о переменной

Строковые функции

addslashes - закавычивает строку слэшами
bin2hex - конвертирует двоичные данные в 16-ричное представление
convert_cyr_string - конвертирует из одного набора символов кириллицы в другой
count_chars - возвращает информацию о символах , используемых с строке
crypt - однонаправленное шифрование строки (хэширование)
echo - выводит одну или более строк
explode - делит строку строкой
htmlentities - конвертирует все символы в HTML-мнемоники
htmlspecialchars - конвертирует специальные символы в HTML-мнемоники
implode - объединяет элементы массива со строкой
join - объединяет элементы массива со строкой
ltrim - вырезает пробелы в начале строки
md5_file - вычисляет md5-хэш данного filename
md5 - вычисляет md5-хэш строки
nl2br - вставляет HTML-разрывы строк (< br />) перед всеми символами \n в строке
parse_str - разбирает строку в переменных
print - выводит строку
printf - выводит отформатированную строку
rtrim - вырезает пробелы в конце строки
setlocale - устанавливает локальную информацию
sprintf - возвращает форматированную строку
sscanf - разбирает ввод из строки в соответствии с форматом
str_repeat - повторяет строку
str_replace - заменяет все вхождения строки поиска строкой замещения
strcasecmp - бинарно безопасное сравнение строк без учета регистра
strchr - находит первое вхождение символа
strcmp - бинарно безопасное сравнение строк
strcoll - сравнение строк на основе установок локализации
strip_tags - вырезает тэги HTML и PHP из строки
stripslashes - раскавычивает строку, закавыченную функцией addslashes()
stristr - strstr() без учёта регистра

strlen - получает длину строки
strncmp - бинарно безопасное сравнение первых n символов строки
strpos - находит позицию первого появления/вхождения в строке
strrchr - находит последнее появление символа в строке
strrev - разворачивает строку
strrpos - находит позицию последнего появления символа в строке
strspn - находит длину начального сегмента, совпадающего с маской
strstr - находит первое появление строки
strtolower - переводит символы строки в нижний регистр
strtoupper - переводит символы строки в верхний регистр
substr_count - подсчитывает количество вхождений подстроки
substr_replace - заменяет текст в части строки
substr - возвращает часть строки
trim - вырезает пробелы в начале и в конце строки
ucfirst - переводит первый символ строки в верхний регистр
ucwords - переводит в верхний регистр первый символ каждого слова строки
vprintf - выводит отформатированную строку
vsprintf - возвращает отформатированную строку

Функции массива

array_chunk - делит массив на отрезки
array_count_values - возвращает количество значений в массиве
array_diff - вычисляет разницу между массивами
array_fill - заполняет массив значениями
array_flip - меняет местами ключи и значения массива
array_intersect - вычисляет пересечение массивов
array_key_exists - проверяет, существует ли данный ключ или индекс в массиве
array_keys - возвращает все ключи массива
array_map - применяет обратный вызов/callback к элементам данного массива
array_merge - объединяет два или более массива
array_multisort - сортирует несколько (или многомерные) массивов
array_pop - выталкивает элемент в конце массива
array_push - ставляет один или более элементов в конец массива
array_rand - извлекает одно или несколько произвольных вхождений из массива
array_reverse - возвращает массив с элементами в обратном порядке
array_search - ищет в массиве значение и возвращает соответствующий ключ при успехе
array_shift - сдвигает элемент за пределы начала массива
array_slice - извлекает отрезок массива
array_splice - удаляет часть массива и заменяет её чем-нибудь
array_sum - высчитывает сумму значений массива
array_unique - удаляет дублирующие значения из массива
array_unshift - присоединяет один или более элементов в начало массива
array_values - возвращает все значения массива
array_walk - применяет пользовательскую функцию к каждому члену массива
array - создаёт массив
arsort - сортирует массив в реверсном порядке учитывая ассоциацию индексов
asort - сортирует массив учитывая ассоциацию индексов
compact - создаёт массив из переменных и их значений
count - подсчитывает элементы в переменной

current - возвращает текущий элемент массива
each - возвращает текущую пару key/value из массива и передвигает курсор массива
end - устанавливает внутренний указатель массива на последний элемент
in_array – возвращает true, если значение существует в массиве
key - извлекает а ключ/key из ассоциативного массива
krsort - сортирует массив по ключам в реверсном порядке
ksort - сортирует массив по ключам
list - присваивает переменные, как если бы они были массивами
next - передвигает вперёд внутренний указатель массива
pos - получает текущий элемент массива
prev - возвращает в начало внутренний указатель массива
range - создаёт массив, содержащий диапазон элементов
reset - устанавливает внутренний указатель массива на первый элемент
rsort - сортирует в реверсном порядке
shuffle - перемешивает массив
sizeof - получает количество элементов в переменной
sort - сортирует массив
uasort - сортирует массив с использованием пользовательской функции сравнения учитывая ассоциацию индексов
uksort - сортирует массив по ключам, используя пользовательскую функцию сравнения
usort - сортирует массив по значениям, используя пользовательскую функцию сравнения

Функции почты

mail - отправляет почту

Функции директории

chdir - сменить директорию
closedir - закрывает дескриптор директории
getcwd - получает текущую рабочую директорию
opendir - открывает дескриптор директории
readdir - читает вхождение из дескриптора директории
rewinddir - переходит в начало дескриптора директории

Функции файловой системы

basename - возвращает компонент filename пути
chmod - изменяет file mode
copy - копирует файл
dirname - возвращает компонент directory name пути
disk_free_space - возвращает доступное пространство в директории
disk_total_space - возвращает общий размер директории
fclose - закрывает указатель открытого файла
feof - проверяет, находится ли указатель файла в точке end-of-file
fflush - очищает вывод в файл
fgetc - получает символ от указателя файла
fgetcsv - получает строку от указателя файла и разбирает CSV-поля
fgets - получает строку от указателя файла
fgetss - получает строку от указателя файла и вырезает тэги HTML
file_exists - проверяет, существует ли файл

file_get_contents - читает весь файл в строку
file - читает весь файл в массив
fileatime - получает время последнего доступа к файлу
filectime - получает время изменения файла
filemtime - получает время последнего изменения файла
filesize - получает размер файла
filetype - получает тип файла
flock - переносимая информирующая блокировка файла
fopen - открывает файл или URL
fputs - записывает в указатель файла
fread - Binary-safe/безопасное в двоичном режиме чтение файла
fscanf - разбирает ввод из файла в соответствии с форматом
fseek - ищет в указателе файла
fstat - получает информацию о файле с использованием указателя открытого файла
ftell - сообщает позицию чтения/записи указателя файла
ftruncate - усекает файл до заданного размера
fwrite - Binary-safe запись в файл
glob - находит пути к файлам, совпадающие с шаблоном
is_dir - сообщает, является ли filename директорией
is_executable - сообщает, является ли filename исполняемым
is_file - сообщает, является ли filename обычным файлом
is_link - сообщает, является ли filename символической ссылкой
is_readable - сообщает, является ли filename читаемым
is_uploaded_file - сообщает, был ли файл загружен через HTTP POST
is_writable - сообщает, можно ли записывать в filename
is_writeable - сообщает, можно ли записывать в filename
link - создаёт жёсткую ссылку
mkdir - создаёт директорию
move_uploaded_file - перемещает загруженный файл в новое место
parse_ini_file - разбирает файл конфигурации
pathinfo - возвращает информацию о пути файла
readfile - выводит файл
realpath - возвращает канонизированный абсолютный pathname
rename - переименовывает файл
rewind - возвращает в начало указатель файла
rmdir - удаляет директорию
stat - выдаёт информацию о файле
symlink - создаёт символическую ссылку
tmpfile - создаёт временный файл
unlink - удаляет файл

Функции управления выводом

flush - выводит и очищает системный буфер вывода
ob_clean - очищает (удаляет) буфер вывода
ob_end_clean - очищает (удаляет) буфер вывода и выключает буферизацию вывода
ob_end_flush - очищает (отправляет) буфер вывода и выключает буферизацию вывода
ob_flush - очищает (отправляет) буфер вывода
ob_get_contents - возвращает содержимое буфера вывода
ob_get_length - возвращает длину буфера вывода

ob_start - включает буферизацию вывода

Функции изображений

Предопределённые константы

IMG_GIF (integer)
IMG_JPG (integer)
IMG_JPEG (integer)
IMG_PNG (integer)
IMG_COLOR_TILED (integer)
IMG_COLOR_STYLED (integer)
IMG_COLOR_BRUSHED (integer)
IMG_COLOR_STYLEDBRUSHED (integer)
IMG_COLOR_TRANSPARENT (integer)
IMG_ARC_ROUNDED (integer)
IMG_ARC_PIE (integer)
IMG_ARC_CHORD (integer)
IMG_ARC_NOFILL (integer)
IMG_ARC_EDGED (integer)

exif_imagetype - определяет тип изображения
getimagesize - получает размер изображения
imagealphablending - устанавливает режим смешения/blending изображения
imagearc - прорисовывает дугу
imagechar - прорисовывает символ горизонтально
imagecharup - прорисовывает символ вертикально
imagecolorallocate - выделяет цвет для изображения
imagecolorat - получает индекс цвета пиксела
imagecolordeallocate - снимает выделение/размещение цвета для изображения
imagecolorexact - получает индекс специфицированного цвета
imagecolorexactalpha - получает индекс специфицированного цвета + alpha
imagecolorstotal - определяет количество цветов в палитре изображения
imagecolortransparent - определяет цвет как прозрачный/transparent
imagecopy - копирует часть изображения
imagecopymerge - копирует и объединяет часть изображения
imagecopymergegray - копирует и объединяет часть изображения с серой шкалой
imagecopyresampled - копирует часть изображения и изменяет его размер путём пересэмплирования
imagecopyresized - копирует часть изображения и изменяет его размер
imagecreate - создаёт новую палитру на основе изображения
imagecreatefromgif - создаёт новое изображение из файла или URL
imagecreatefromjpeg - создаёт новое изображение из файла или URL
imagecreatefrompng - создаёт новое изображение из файла или URL
imagecreatetruecolor - создаёт новое изображение true color
imagedashedline - прорисовывает пунктирную линию
imagedestroy - разрушает изображение
imageellipse - прорисовывает эллипс
imagefill - заполняет заливкой
imagefilledarc - прорисовывает часть эллипса и заполняет её
imagefilledellipse - прорисовывает заполненный эллипс

imagefilledpolygon - прорисовывает заполненный многоугольник
imagefilledrectangle - прорисовывает заполненный прямоугольник
imagefilltoborder - заполняет заливкой специфицированного цвета
imagefontheight - получает высоту шрифта
imagefontwidth - получает ширину шрифта
imagegif - выводит-изображение в браузер или файл
imagejpeg - выводит-изображение в браузер или файл
imageline - прорисовывает линию
imageloadfont - загружает новый шрифт
imagepalettecopy - копирует палитру из одного изображения в другое
imagepng - выводит PNG-изображение в браузер или файл
imagepolygon - прорисовывает многоугольник
imagerectangle - прорисовывает прямоугольник
imagesetbrush - устанавливает изображение кисти для прорисовки линии
imagesetpixel - устанавливает одиночный пиксел
imagesetthickness - устанавливает толщину прорисовки линии
imagestring - прорисовывает строку горизонтально
imagestringup - прорисовывает строку вертикально
imagesx - получает ширину изображения
imagesy - получает высоту изображения
imagetruecolortopalette - конвертирует изображение true color в палитровое/palette изображение

Прочие функции

define - определяет именованную константу
defined - проверяет, существует ли данная именованная константа
die - псевдоним exit()
eval - вычисляет строку как PHP-код
exit - выводит сообщение и прерывает выполнение текущего скрипта
get_browser - сообщает, какой браузер у пользователя
sleep - задержать выполнение
uniqid - генерирует уникальный id
usleep - задержка выполнения в микросекундах

Функции регулярных выражений

preg_grep - возвращает вхождения массива, совпадающие с шаблоном
preg_match_all - выполняет глобальный поиск совпадений регулярного выражения
preg_match - выполняет поиск совпадений регулярного выражения
preg_quote - закавычивает символы регулярного выражения
preg_replace_callback - выполняет поиск регулярного выражения и замену с использованием функции обратного вызова/callback
preg_replace - выполняет поиск регулярного выражения и замену
preg_split - делит строку регулярным выражением
split - делит строку на массив с помощью регулярного выражения
spliti - делит строку на массив с помощью регулярного выражения без учёта регистра символов

Задание к лабораторной работе:

1. Установить программу Denwer (или любой другой локальный веб-сервер), включающую в себя web-сервер Apache, язык программирования PHP и СУБД MySQL.
2. Переместить сайт, разработанный в лабораторной работе №2, в директорию /WebServers/home/имя_сайта/www/ и перезапустить Denwer. Изменить сайт, используя возможности языка программирования PHP.
3. Проанализировать сайт и вынести общие элементы web-страниц, такие как шапка (header), подвал (footer), меню и т.д. в отдельные файлы, подключив их к основным скриптам функциями require, require_once, include, include_once. При этом, такое содержимое шапки, как заголовок страницы(тег `<title>`), тег `<meta>` и др. на разных страницах должно отличаться, т.е. соответствовать назначению текущей страницы.
4. Средствами языка PHP произвести проверку данных, введенных в поля формы (на заполнение, на правильность email, на ввод только числового значения и т.д. при помощи регулярных выражений). В случае ошибок вывести их, выделить неправильно заполненные поля (бэграундом, цветом, картинкой) и заполнить поля теми же значениями, которые там были до отправки формы. Использовать метод POST.
5. Сделать страничку на PHP для подсчета размера, занимаемого на жестком диске любой папкой или файлом вашего сайта (можно разместить на about.php). Наименование папки/файла должно передаваться методом GET в виде относительного пути. *Предусмотреть ограничения на ввод, учитывая, что скрипту должна быть доступна только корневая папка сайта и вложенные.*
6. Реализовать одно из следующих заданий, разработав класс для обработки данных:
   - *отправка email'а через форму обратной связи с полями «Представьтесь», «Ваш E-mail», «Текст сообщения». Использовать несколько дополнительных заголовков и кодировку UFT-8. Данные полей должны проверяться на правильность.*
   - *RSS-поток новостей с возможностью просмотра в RSS-агрегаторе или браузере. Данные для потока читать из файла. Формат хранимых в файле данных разработать самостоятельно.*
   - *наложение текстов/изображений на другие изображения (водяные знаки), загружаемые с компьютера пользователя, с отображением ссылки на вновь полученный файл.*
   - *рисование диаграмм по хранимым в произвольном файле данным. Вид диаграммы должен выбираться пользователем (график, гистограмма).*
   - *организовать в файле локальную базу данных по определенной предметной области (например база данных аудиотреков, видеофильмов, компьютерных комплектующих и т.д.) с функциями сохранения, чтения и удаления. Список хранимых атрибутов (полей) определить самостоятельно.*
   - *из заданного статичного html-файла со списком новостей (можно сохранить любую html-страницу с новостями) используя регулярные выражения в формате PCRE вывести все новости на web-страницу игнорируя ненужную служебную информацию, такую как строка меню, шапка заголовка, подвал, т.е. все что не является новостями.*
   - *ведение статистики по посетителям сайта (ip-адрес, название и версия браузера, количество посещений, заходов на конкретную страницу, дата/время входа). Данные сохранять в *.csv файл и представить на PHP-странице в*

удобном виде с возможностью группировки данных и выводом итоговых значений. Предусмотреть отображение в различных разрезах (по времени, по датам, по браузеру, по ip-адресу) в процентных соотношениях.

7. Страницы должны быть работоспособны в любом браузере.

Каждый отчет должен содержать:

1. Заголовок лабораторной работы (название и цель работы).
2. Фамилия, инициалы и группа студента.
3. Задание к лабораторной работе.
4. Краткие теоретические сведения.
5. Описание алгоритмов, функций, примененных решений.
6. Экранные формы разработанных страниц.
7. Основные тексты страниц.
8. Выводы по сделанной работе.